

SQL Filtering Practice: 12 Queries

Operators covered: WHERE, IN, BETWEEN, LIKE | Dataset: Northwind-style (7 tables) | 29 September 2026

Objective

Practice common SQL filters on the same cleaned dataset used in the last project. Each query tests one operator at a time — WHERE with comparisons, IN for a list of values, BETWEEN for a range, and LIKE for pattern matching — so it's clear what each one does on its own before combining them.

Queries and outputs

Query 1: Products priced between 15 and 30 (BETWEEN)

```
SELECT ProductName, Price
FROM Products
WHERE Price BETWEEN 15 AND 30
ORDER BY Price;
```

ProductName	Price
Tomato Sauce	15.00
Peanut Butter	15.00
Granola Bars	16.00
Chais	18.00
Chocolate Sauce	18.00
Chang	19.00
Vegetable Stir-Fry Mix	19.99
Chef Anton's Gumbo Mix	21.35
Chef Anton's Cajun Seasoning	22.00
Clam Chowder	22.00
Jasmine Rice	25.00
Maple Syrup	25.00
Grilled Chicken	30.00
Vanilla Extract	30.00

14 row(s) returned.

Query 2: Customers from three specific countries (IN)

```
SELECT CustomerName, Country
FROM Customers
WHERE Country IN ('Germany', 'France', 'Mexico')
ORDER BY Country;
```

CustomerName	Country
Blondel père et fils	France
Bon app'	France
Chocolaterie Richart	France
Dolores's Fine Foods	France
Vincent's Wine Bar	France
Alfreds Futterkiste	Germany
Blauer See Delikatessen	Germany
Feinkost Käfer	Germany
Gourmet Delights	Germany
Ana Trujillo Emparedados y helados	Mexico
Antonio Moreno Taquería	Mexico

11 row(s) returned.

Query 3: Products NOT in a set of categories (NOT IN)

```
SELECT ProductName, CategoryID
FROM Products
WHERE CategoryID NOT IN (1, 2)
ORDER BY CategoryID;
```

ProductName	CategoryID
Granola Bars	3
Grilled Chicken	4
Cream Cheese	4
Whole Wheat Pasta	5
Organic Oats	5
Jasmine Rice	5
Almond Flour	5

7 row(s) returned.

Query 4: Customers whose name ends in 's' (LIKE)

```
SELECT CustomerName, City
FROM Customers
WHERE CustomerName LIKE '%s'
ORDER BY CustomerName;
```

CustomerName	City
Ana Trujillo Emparedados y helados	México D.F.
Blondel père et fils	Strasbourg
Dolores's Fine Foods	Paris
Eurasian Delicacies	Beijing
Fiesta Foods	Santiago
Gourmet Delights	Berlin
Jamaica Imports	Kingston
South American Treats	Buenos Aires
Tokyo Delicacies	Tokyo

9 row(s) returned.

Query 5: Products with 'Cha' anywhere in the name (LIKE wildcard both sides)

```
SELECT ProductName, Price
FROM Products
WHERE ProductName LIKE '%Cha%';
```

ProductName	Price
Chais	18.00
Chang	19.00

2 row(s) returned.

Query 6: Suppliers NOT from the USA (WHERE + <>)

```
SELECT SupplierName, Country
FROM Suppliers
WHERE Country <> 'USA'
ORDER BY Country;
```

SupplierName	Country
Jasmine Tea Co.	China
Herbs and Spices	India
Luxury Chocolates	India
Dolce Sweets	Italy
Mediterranean Bites	Italy
Blue Mountain Coffee	Jamaica
Tokyo Traders	Japan
African Spice Traders	Nigeria
Cooperativa de Quesos 'Las Cabras'	Spain
Old World Olive Oil	Spain
Vintage Vineyards	Spain
Exotic Liquids	UK
Urban Artisan Bakery	UK

13 row(s) returned.

Query 7: Orders from a specific set of shippers (IN)

```
SELECT OrderID, CustomerID, ShipperID
FROM Orders
WHERE ShipperID IN (1, 2)
ORDER BY OrderID;
```

OrderID	CustomerID	ShipperID
10249	81	1
10250	34	2
10251	88	1
10252	76	2
10254	89	1
10255	12	2
10256	73	1
10257	22	2
10259	45	1
10261	39	1
10262	41	2
10263	54	1
10264	66	2
10266	33	1
10268	25	2

15 row(s) returned.

Query 8: Orders placed in a date range (BETWEEN on dates)

```
SELECT OrderID, OrderDate
FROM Orders
WHERE OrderDate BETWEEN '2022-07-01' AND '2022-07-10'
ORDER BY OrderDate;
```

OrderID	OrderDate
10248	2022-07-04
10249	2022-07-05
10250	2022-07-08
10251	2022-07-08
10252	2022-07-09
10253	2022-07-10

6 row(s) returned.

Query 9: Order lines with quantity between 5 and 10 (BETWEEN)

```
SELECT OrderID, ProductID, Quantity
FROM OrderDetails
WHERE Quantity BETWEEN 5 AND 10
ORDER BY Quantity;
```

OrderID	ProductID	Quantity
10248	72	5
10255	33	5
10259	54	6
10253	47	7
10262	45	7
10250	25	8
10263	56	8
10249	14	9
10253	35	9
10266	49	9
10248	42	10
10252	61	10

12 row(s) returned.

Query 10: Products priced under 10 OR over 50 (WHERE + OR)

```
SELECT ProductName, Price
FROM Products
WHERE Price < 10 OR Price > 50
ORDER BY Price;
```

ProductName	Price
Cream Cheese	2.70
Whole Wheat Pasta	3.50
Organic Oats	4.50
Basil Pesto	7.20
Instant Coffee	9.50
Mango Chutney	9.95

6 row(s) returned.

Query 11: Customers whose City starts with 'M' (LIKE)

```
SELECT CustomerName, City
FROM Customers
WHERE City LIKE 'M%'
ORDER BY City;
```

CustomerName	City
Delicias de España	Madrid
Blauer See Delikatessen	Mannheim
Ana Trujillo Emparedados y helados	México D.F.
Antonio Moreno Taquería	México D.F.
Feinkost Käfer	München

5 row(s) returned.

Query 12: Categories with an ID in a chosen list (IN)

```
SELECT CategoryID, CategoryName
FROM Categories
WHERE CategoryID IN (1, 3, 5);
```

CategoryID	CategoryName
1	Beverages
3	Confections
5	Grains/Cereals

3 row(s) returned.

What I noticed testing each operator

- **IN** is a shorter way to write several OR conditions on the same column — Country IN ('Germany', 'France', 'Mexico') instead of three OR clauses.
- **BETWEEN** is inclusive on both ends, so Price BETWEEN 15 AND 30 includes products priced at exactly 15 or 30.
- **LIKE** needs a wildcard: % matches any number of characters, so '%s' finds names ending in 's', and '%Cha%' finds the text anywhere in the name.
- **NOT IN** and <> are useful for excluding values instead of including them.

Interview questions

LIKE vs =? = matches a value exactly. LIKE matches a pattern using wildcards (% for any characters, _ for a single character), so it's used for partial or fuzzy text matches, not exact ones.

When to use BETWEEN? When filtering a column for a continuous range, like a price range, a date range, or a quantity range. It's inclusive of both endpoints and reads more clearly than writing `column >= low AND column <= high`.